TIMBRE — DOCUMENTAÇÃO COMPLETA DO PROJETO

Etapa 1 (Proposta Revisada) · Etapa 2 (Modelagem UML e Dicionário) · Etapa 3 (Implementação, CRUD, Telas e Testes)

Campo	Informações do Grupo e da Entrega
Nome do Projeto	Timbre — Conexão entre músicos e bandas locais por vagas compatíveis
Grupo	Grupo 16
Integrantes	Iago Boettge · Marcus Vinicius Rodrigues de Jesus · Samuel Almeida Giacomelli · Fernanda Silva Bettanin
Repositório Remoto (GitHub)	https://github.com/IagoBoettge/Timbre
Linguagem e Interface	Python 3.10+ (verificado em Python 3.12.14) + Tkinter (ttk) + Persistência JSON
Data da Entrega e Demonstração	05/11/2026 (Aula de demonstração funcional do código)

1. Etapa 1 — Proposta do Projeto (Revisada e Atualizada)

1.1. Dados do Grupo e Ambiente de Desenvolvimento

O projeto Timbre foi desenvolvido pelo Grupo 16 (Iago Boettge, Marcus Vinicius Rodrigues de Jesus, Samuel Almeida Giacomelli e Fernanda Silva Bettanin). As ferramentas e tecnologias adotadas no ambiente de desenvolvimento foram:

- **GitHub:** controle de versão, colaboração por branches/Pull Requests e hospedagem do código-fonte.
- **Diagramas UML (draw.io / Mermaid):** modelagem orientada a objetos (diagrama de classes, dicionário de classes e arquitetura de interação).
- **Python 3 (Biblioteca Padrão):** implementação das regras de domínio, validações, testes automatizados (unittest) e persistência em arquivo JSON.
- **Tkinter / tkinter.ttk:** construção da interface gráfica desktop organizada em abas e tabelas interativas.

1.2. Problema e Proposta do Projeto

Problema a ser resolvido: A dificuldade de encontrar integrantes compatíveis para bandas locais e, na visão do músico, de localizar grupos da sua região que precisem exatamente do seu instrumento e nível de experiência.

Proposta revisada: Criação de uma aplicação desktop em Python + Tkinter na qual músicos cadastram seus dados pessoais, gêneros de interesse e habilidades musicais (instrumento e nível de 1 a 3), enquanto bandas locais registram seu perfil, integrantes e vagas abertas com instrumento procurado, nível mínimo e requisitos. O sistema cruza automaticamente as habilidades do músico com as vagas abertas e aplica filtros opcionais por cidade, instrumento e gênero musical.

1.3. Requisitos — Evolução das Classes do Projeto

Na concepção inicial da Etapa 1 foram definidas 5 classes principais (Musico, HabilidadeMusical, Busca, Vaga e Banda). Na revisão para a Etapa 2 e Etapa 3, o modelo foi refinado para 8 classes, incorporando herança/polimorfismo (Perfil), persistência CRUD (Cadastro) e camada de interface gráfica (Aplicacao):

Classe	Origem	Responsabilidade no Sistema
Perfil (abstrata)	Etapa 2 (UML)	Classe base abstrata (ABC) com id, nome, cidade, validação protegida (_validarDados) e contrato abstrato resumirPerfil().
Musico	Etapa 1 / Etapa 2	Herda de Perfil; armazena idade, gêneros de interesse e lista de objetos HabilidadeMusical. Implementa resumirPerfil().
Banda	Etapa 1 / Etapa 2	Herda de Perfil; armazena estilo musical, lista de integrantes (Musico) e vagas publicadas (Vaga). Implementa resumirPerfil().
HabilidadeMusical	Etapa 1 / Etapa 2	Representa o instrumento que o músico toca (ou vocal) e seu nível de domínio (1 = iniciante, 2 = intermediário, 3 = avançado).
Vaga	Etapa 1 / Etapa 2	Identifica a oportunidade publicada por uma banda (instrumento, nível mínimo de 1 a 3, requisitos e estado aberta/encerrada).
Busca	Etapa 1 / Etapa 2	Filtra vagas abertas e compatíveis com as habilidades do músico, aplicando filtros opcionais de cidade, instrumento e gênero.
Cadastro	Etapa 3 (CRUD)	Gerencia os dicionários em memória por ID (músicos, bandas e vagas), integridade referencial nas exclusões e persistência em JSON.
Aplicacao	Etapa 3 (GUI)	Constrói a janela Tkinter com 5 abas (Músicos, Bandas, Vagas, Integrantes e Buscar oportunidades) e conecta eventos ao Cadastro e Busca.

1.4. Entrada e Saída de Dados

• Entradas de dados do sistema:

- **Músico:** Nome, cidade, idade, gêneros musicais de interesse e habilidades musicais (instrumento e nível 1, 2 ou 3).
- **Banda:** Nome da banda, cidade, estilo musical e associação de integrantes pelo ID.
- **Vaga:** ID da banda responsável, instrumento procurado, nível mínimo de experiência (1 a 3) e requisitos em texto livre.
- **Busca:** ID do músico e filtros opcionais de cidade desejada, instrumento e gênero musical.

• Saídas de dados do sistema:

- Listagem formatada dos músicos cadastrados com resumo polimórfico do perfil e habilidades detalhadas.
- Listagem das bandas cadastradas, estilo musical, cidade e quantidade/nomes dos integrantes vinculados.
- Listagem das vagas com banda responsável, instrumento, nível mínimo, estado (Aberta/Encerrada) e requisitos.
- Tabela de oportunidades compatíveis na aba de busca, exibindo banda, cidade, estilo, instrumento, nível exigido e requisitos.
- Mensagens de status no rodapé e caixas de diálogo (messagebox) informando erros de validação ou confirmação de exclusão.

1.5. Principais Funcionalidades do Programa

- **1. Cadastro de músicos:** permite informar nome, cidade, idade, gêneros musicais e múltiplos instrumentos com nível de 1 a 3.
- **2. Cadastro de bandas:** permite registrar nome, cidade e estilo musical principal da banda.
- **3. Gerenciamento de integrantes:** permite vincular e remover músicos integrantes de cada banda pelos seus IDs.
- **4. Cadastro de vagas:** permite que bandas publiquem vagas informando instrumento, nível mínimo desejado e requisitos opcionais.
- **5. Busca de bandas e vagas compatíveis:** cruza automaticamente as habilidades e níveis do músico com as vagas abertas.
- **6. Filtro por cidade:** restringe as oportunidades à cidade informada (ignorando maiúsculas, minúsculas e acentos).
- **7. Filtro por habilidade/instrumento:** permite focar a busca em um instrumento específico que o músico toca.
- **8. Filtro por gênero musical:** permite filtrar bandas que toquem um estilo específico (rock, pop, jazz, samba, metal, etc.).
- **9. Visualização de oportunidades:** exibe em tabela todas as informações da banda, vaga, nível mínimo e requisitos.
- **10. Atualização e exclusão (CRUD completo):** permite editar ou remover músicos, bandas e vagas mantendo a integridade dos vínculos.
- **11. Encerramento e reabertura de vagas:** permite alternar o estado da vaga entre Aberta e Encerrada sem perder seu histórico.
- **12. Persistência automática em JSON:** grava localmente cada alteração em data/timbre.json e recarrega ao abrir o aplicativo.

1.6. Cronograma e Divisão de Responsabilidades

Todo o grupo é responsável pelo projeto e por todas as suas partes. Para organizar a execução das três etapas, as atividades principais foram distribuídas conforme a matriz abaixo:

Tarefa / Atividade	Iago Boettge	Marcus Vinicius	Fernanda Silva	Samuel Almeida
Criação da UML e Dicionário de Classes		X		
Criação das classes de domínio	X			
Codificação (Python)	X	X	X	X
Identidade visual e captura de telas			X	
Implementação da interface Tkinter				X
Criação e gestão do repositório GitHub			X	X
Polimorfismo e Atributo privado		X		
Herança (classe abstrata Perfil)				X
Associação e Método privado	X			
Testes do programa (unitários e GUI)			X	
Elaboração dos documentos das etapas	X	X	X	X

2. Etapa 2 Completa — Modelagem UML, Dicionário de Classes e Arquitetura

Esta seção consolida integralmente o documento da Etapa 2 (Diagrama UML - Grupo 16.docx), composto por três pranchas (Diagrama de Classes UML, Dicionário das Classes e Arquitetura de Entradas/Saídas/Interação), seguido pelo Diagrama de Classes UML Atualizado na Etapa 3 que reflete todas as adições da implementação.

2.1. Diagrama de Classes Original (Etapa 2 — Página 1)

O diagrama original da Etapa 2 estabeleceu a hierarquia encabeçada pela superclasse abstrata Perfil, herdada por Musico e Banda, além das classes HabilidadeMusical, Vaga e Busca, definindo visibilidades (+ público, - privado, # protegido), multiplicidades (1, 0..*, 1..*) e dependências («use»).

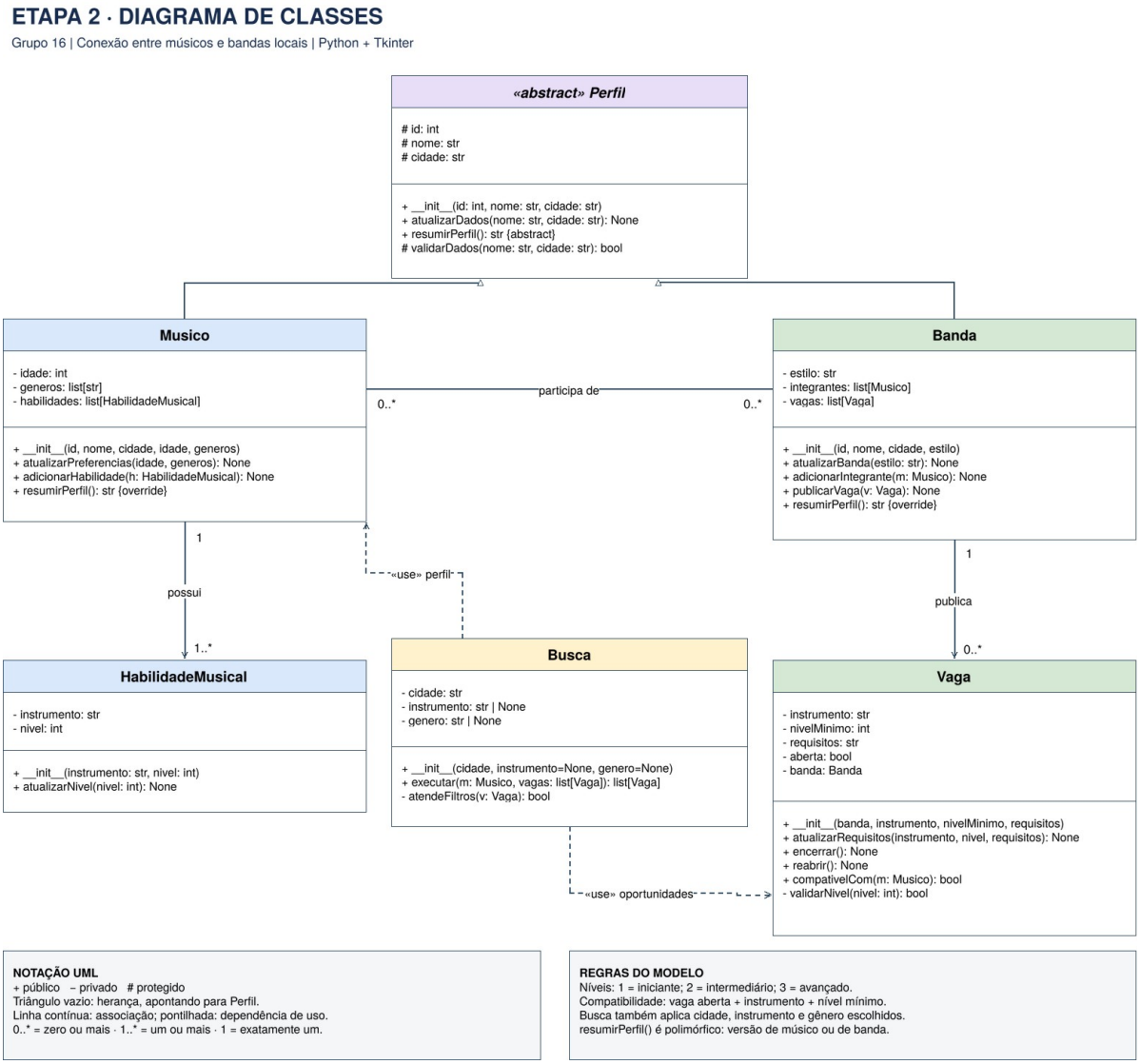
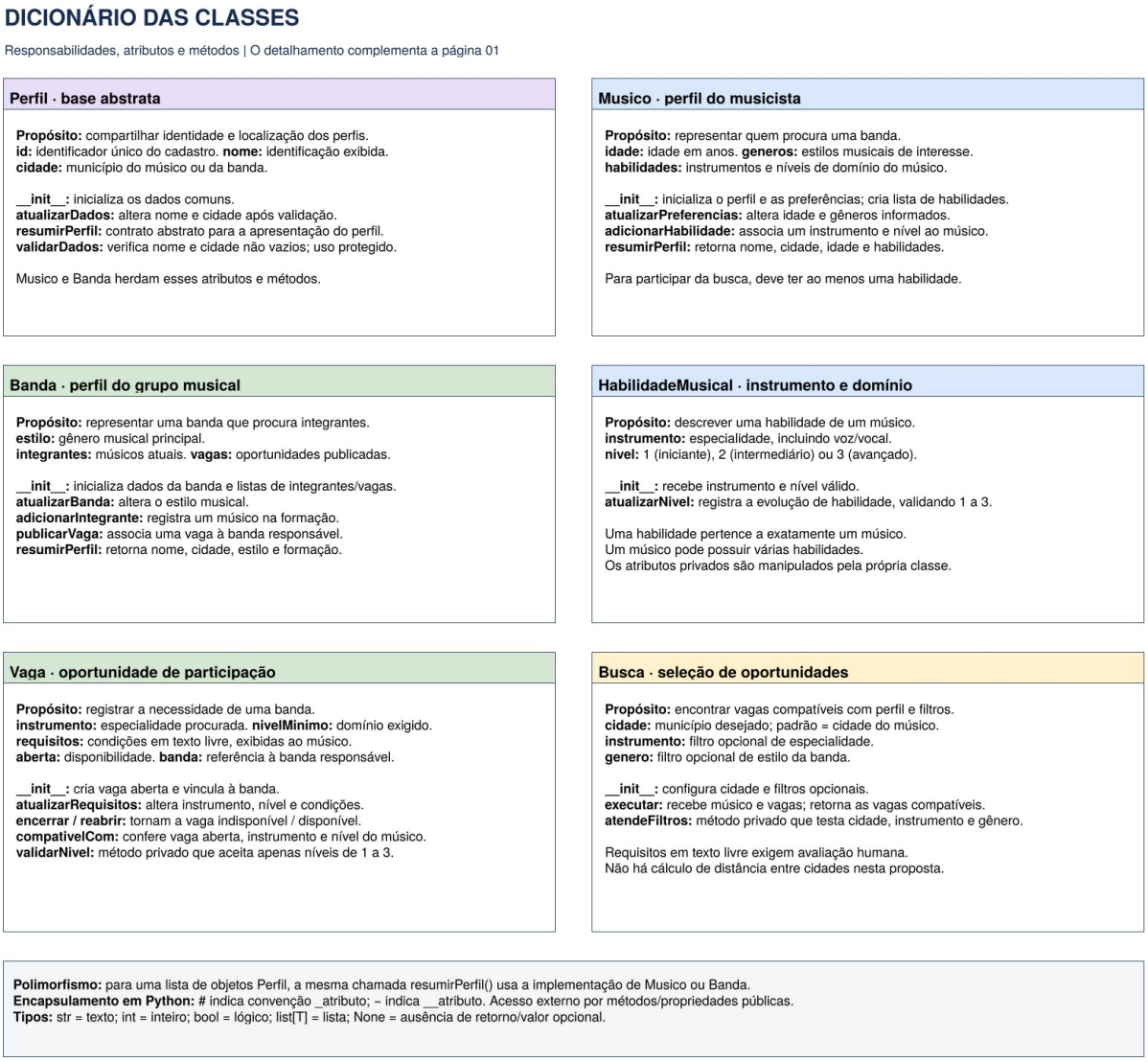


Figura 1 — Diagrama de Classes UML original elaborado na Etapa 2 (Grupo 16).

2.2. Dicionário das Classes (Etapa 2 — Página 2)

O Dicionário das Classes detalha o propósito, os atributos, os métodos e as regras de negócio de cada uma das 6 classes do modelo de domínio concebido na Etapa 2:



DICIONÁRIO DAS CLASSES

Responsabilidades, atributos e métodos | O detalhamento complementa a página 01

Perfil · base abstrata

Propósito: compartilhar identidade e localização dos perfis.
id: identificador único do cadastro. **nome:** identificação exibida.
cidade: município do músico ou da banda.

__init__: inicializa os dados comuns.
atualizarDados: altera nome e cidade após validação.
resumirPerfil: contrato abstrato para a apresentação do perfil.
validarDados: verifica nome e cidade não vazios; uso protegido.

Musico e Banda herdam esses atributos e métodos.

Musico · perfil do musicista

Propósito: representar quem procura uma banda.
idade: idade em anos. **generos:** estilos musicais de interesse.
habilidades: instrumentos e níveis de domínio do músico.

__init__: inicializa o perfil e as preferências; cria lista de habilidades.
atualizarPreferencias: altera idade e gêneros informados.
adicionarHabilidade: associa um instrumento e nível ao músico.
resumirPerfil: retorna nome, cidade, idade e habilidades.

Para participar da busca, deve ter ao menos uma habilidade.

Banda · perfil do grupo musical

Propósito: representar uma banda que procura integrantes.
estilo: gênero musical principal.
integrantes: músicos atuais. **vagas:** oportunidades publicadas.

__init__: inicializa dados da banda e listas de integrantes/vagas.
atualizarBanda: altera o estilo musical.
adicionarIntegrante: registra um músico na formação.
publicarVaga: associa uma vaga à banda responsável.
resumirPerfil: retorna nome, cidade, estilo e formação.

HabilidadeMusical · instrumento e domínio

Propósito: descrever uma habilidade de um músico.
instrumento: especialidade, incluindo voz/vocal.
nivel: 1 (iniciante), 2 (intermediário) ou 3 (avançado).

__init__: recebe instrumento e nível válido.
atualizarNivel: registra a evolução de habilidade, validando 1 a 3.

Uma habilidade pertence a exatamente um músico.
Um músico pode possuir várias habilidades.
Os atributos privados são manipulados pela própria classe.

Vaga · oportunidade de participação

Propósito: registrar a necessidade de uma banda.
instrumento: especialidade procurada. **nivelMinimo:** domínio exigido.
requisitos: condições em texto livre, exibidas ao músico.
aberta: disponibilidade. **banda:** referência à banda responsável.

__init__: cria vaga aberta e vincula à banda.
atualizarRequisitos: altera instrumento, nível e condições.
encerrar / reabrir: tornam a vaga indisponível / disponível.
compativelCom: confere vaga aberta, instrumento e nível do músico.
validarNivel: método privado que aceita apenas níveis de 1 a 3.

Busca · seleção de oportunidades

Propósito: encontrar vagas compatíveis com perfil e filtros.
cidade: município desejado; padrão = cidade do músico.
instrumento: filtro opcional de especialidade.
genero: filtro opcional de estilo da banda.

__init__: configura cidade e filtros opcionais.
executar: recebe músico e vagas; retorna as vagas compatíveis.
atendeFiltros: método privado que testa cidade, instrumento e gênero.

Requisitos em texto livre exigem avaliação humana.
Não há cálculo de distância entre cidades nesta proposta.

Polimorfismo: para uma lista de objetos Perfil, a mesma chamada resumirPerfil() usa a implementação de Musico ou Banda.
Encapsulamento em Python: # indica convenção _atributo; – indica __atributo. Acesso externo por métodos/propriedades públicas.
Tipos: str = texto; int = inteiro; bool = lógico; list[T] = lista; None = ausência de retorno/valor opcional.

Figura 2 — Dicionário das Classes original da Etapa 2: responsabilidades, atributos e métodos.

2.3. Entradas, Saídas, Interação e Persistência Proposta (Etapa 2 — Página 3)

A terceira parte da Etapa 2 especificou o fluxo de entrada por formulários, as regras de processamento e validação, a saída em tabelas no Tkinter, a estrutura de abas propostas e a decisão arquitetural de persistir os cadastros localmente em arquivo JSON preservando os identificadores entre sessões:

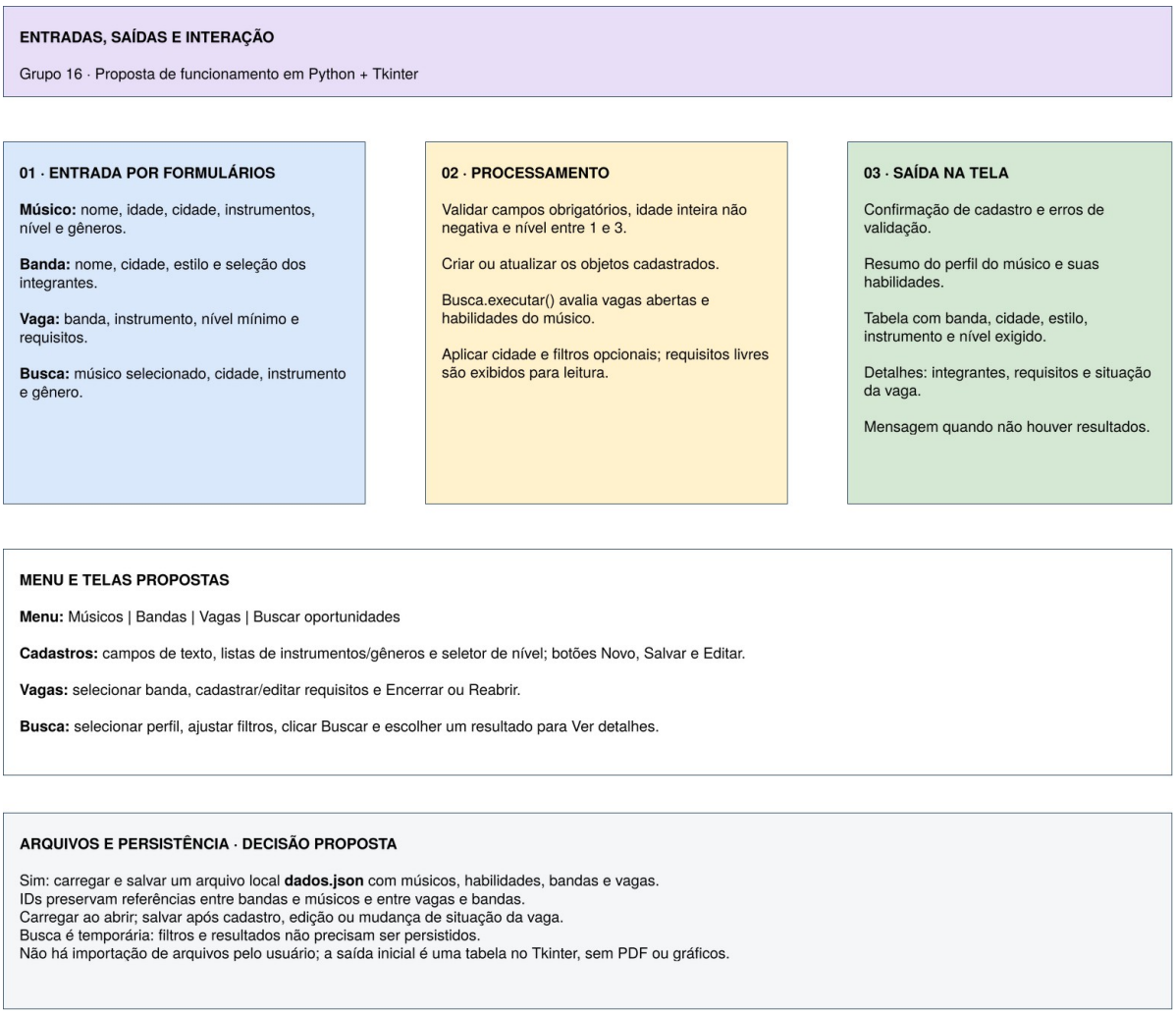


Figura 3 — Especificação de Entradas, Saídas, Interação em Tkinter e Persistência JSON da Etapa 2.

2.4. Diagrama de Classes UML Atualizado (Etapa 3 — Implementação Final)

Durante a codificação da Etapa 3, o modelo original da Etapa 2 foi integralmente respeitado em suas regras e assinaturas de domínio, recebendo extensões pontuais para viabilizar o CRUD completo, a persistência JSON e a interface gráfica Tkinter:

- **Classe Cadastro:** adicionada em dados.py para coordenar os dicionários de músicos, bandas e vagas por ID sequencial, garantir exclusão em cascata e realizar leitura/gravação atômica em data/timbre.json.
- **Classe Aplicacao:** adicionada em app.py para construir as 5 abas Tkinter, gerenciar seleção em tabelas Treeview e restaurar o estado anterior (via copy.deepcopy) caso alguma validação falhe.
- **Métodos removerIntegrante() e removerVaga():** adicionados à classe Banda para permitir desvincular músicos e remover vagas excluídas.
- **Convenção de visibilidade em Python:** em Python, o método protegido de Perfil foi implementado como _validarDados() (convenção #) e o método privado de Vaga como __validarNivel() (name mangling -), exatamente conforme a visibilidade planejada na UML.

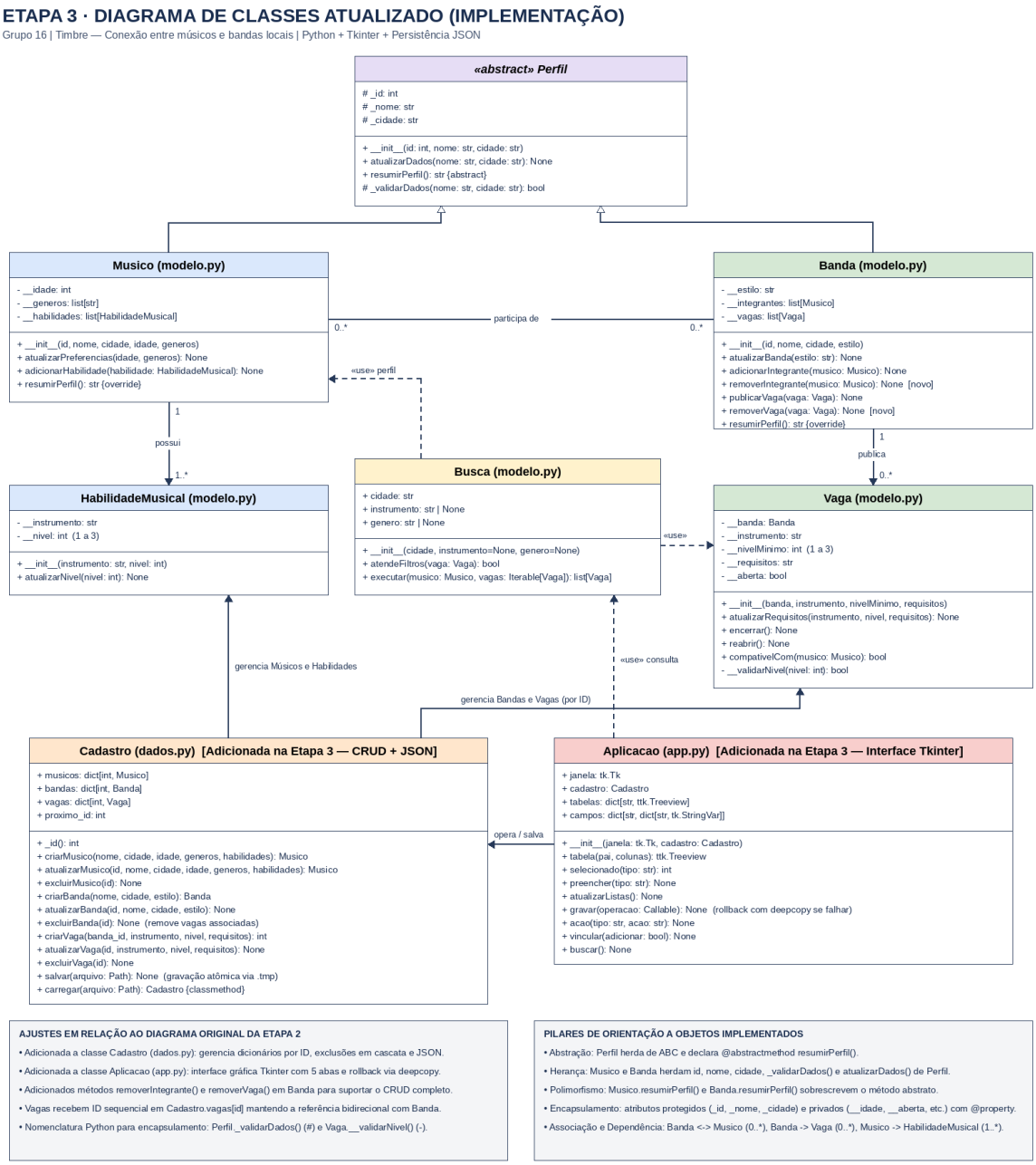


Figura 4 — Diagrama de Classes UML Atualizado na Etapa 3 com as classes Cadastro (dados.py) e Aplicacao (app.py).

2.5. Aplicação dos Pilares de Orientação a Objetos no Código

Conceito de POO	Onde está no código	Como foi implementado
Abstração	modelo.py (linhas 23–51): classe Perfil(ABC)	Perfil herda de ABC (módulo abc) e declara @abstractmethod def resumirPerfil(self), impedindo instanciação direta de Perfil.
Herança	modelo.py (linhas 73 e 110): Musico(Perfil) e Banda(Perfil)	Musico e Banda chamam super().__init__(id, nome, cidade) e herdam as propriedades id, nome, cidade e atualizarDados().
Polimorfismo	modelo.py (linhas 106 e 152) e app.py (linhas 117 e 120)	Musico.resumirPerfil() retorna 'Músico: nome | cidade | idade anos' e Banda.resumirPerfil() retorna 'Banda: nome | cidade | estilo'.
Atributos Protegidos	modelo.py: _id, _nome,	Atributos de Perfil usam prefixo simples (_) para

Conceito de POO	Onde está no código	Como foi implementado
e Privados	_cidade (#) e __idade, __generos, __habilidades, __estilo, __aberta (-)	acesso nas subclasses; atributos internos de cada classe usam duplo sublinhado (__) e @property somente leitura.
Método Protegido e Método Privado	modelo.py: Perfil._validarDados() (linha 40) e Vaga.__validarNivel() (linha 184)	_validarDados() valida nome e cidade na hierarquia; __validarNivel() usa name mangling privado em Vaga para garantir nível inteiro em {1, 2, 3}.
Associação e Dependência	modelo.py e dados.py	Musico compõe HabilidadeMusical (1..*); Banda agrega Musico (0..*) e publica Vaga (0..*); Vaga referencia Banda; Busca usa («use») Musico e Vaga.

3. Repositório Remoto e Estrutura do Código

Link oficial do repositório remoto (GitHub): https://github.com/IagoBoettge/Timbre

Todo o código-fonte funcional da Etapa 3, os testes unitários, o script de demonstração e os artefatos de documentação encontram-se integrados na branch principal (main) do repositório do Grupo 16 no GitHub.

Arquivo	Descrição e Papel na Arquitetura
modelo.py	Implementa as classes de domínio (Perfil, HabilidadeMusical, Musico, Banda, Vaga e Busca), validações, normalização de texto sem acentos e regras de compatibilidade.
dados.py	Implementa a classe Cadastro com todas as operações de CRUD em memória, manutenção de integridade entre objetos e persistência atômica em JSON.
app.py	Implementa a interface gráfica Tkinter (classe Aplicacao) com 5 abas interativas, tabelas Treeview, tratamento de erros com messagebox e salvamento automático.
demo.py	Script de demonstração rápida pelo terminal que executa cadastro, busca, encerramento/reabertura de vaga e persistência JSON em diretório temporário.
test_timbre.py	Suíte com 19 testes automatizados (unittest) cobrindo regras de negócio, filtros, validações, POO, CRUD e integridade do arquivo JSON.
data/timbre.json	Arquivo JSON gerado e atualizado automaticamente pela aplicação para persistir músicos, bandas, vagas e vínculos de integrantes.

Como Executar o Projeto

Requisitos: Python 3.10 ou superior (testado em Python 3.12.14) com Tkinter disponível. Não é necessário instalar pacotes externos via pip.

```
# 1. Iniciar a interface gráfica Tkinter:
python3 app.py

# 2. Executar a demonstração de regras no terminal:
python3 demo.py

# 3. Executar todos os 19 testes automatizados:
python3 -m unittest discover -s . -p "test_*.py" -v
```

4. Explicação das Classes Externas e Bibliotecas Utilizadas

O projeto utiliza exclusivamente módulos e classes da Biblioteca Padrão do Python (Python Standard Library), o que garante portabilidade imediata sem necessidade de instalar bibliotecas de terceiros. A tabela abaixo explica cada biblioteca e classe externa utilizada e onde ela atua no código:

Biblioteca / Módulo	Classes / Funções Importadas	Explicação do Uso no Projeto
tkinter	Tk, StringVar	Cria a janela raiz da aplicação desktop (Tk) e gerencia variáveis reativas (StringVar) vinculadas aos campos de texto dos formulários e à barra de status em app.py.
tkinter.ttk	Notebook, Frame, Label, Entry, Button, Treeview	Fornece os widgets temáticos da interface: Notebook (navegação nas 5 abas), Frame (containers de layout), Label/Entry/Button (formulários e ações) e Treeview (tabelas de listagem e seleção de registros).
tkinter.messagebox	showerror, askyesno	Exibe janelas modais de alerta quando uma validação falha (showerror) e solicita confirmação do usuário antes de excluir registros (askyesno).
abc	ABC, abstractmethod	Usada em modelo.py para declarar Perfil como classe base abstrata (ABC) e exigir que as subclasses Musico e Banda implementem resumirPerfil() (@abstractmethod).
unicodedata	normalize, category	Usada na função chave() em modelo.py para decompor caracteres acentuados (NFD) e remover marcas diacríticas (categoria 'Mn'), permitindo buscas e comparações insensíveis a acentos e caixa ('São Paulo' == 'SAO PAULO').
json	dumps, loads, JSONDecodeError	Usada em dados.py para serializar o estado completo do Cadastro em texto JSON formatado (UTF-8, indent=2) e reconstruir os objetos ao iniciar o programa.
pathlib	Path	Manipulação orientada a objetos dos caminhos de arquivos (data/timbre.json e arquivos temporários .json.tmp) de forma independente de sistema operacional.
os	replace	Usada em Cadastro.salvar() (dados.py) para substituir atomicamente o arquivo data/timbre.json pelo arquivo temporário recém-gravado, evitando corrupção de dados.
copy	deepcopy	Usada em Aplicacao.gravar() (app.py) para criar um snapshot em memória antes de qualquer operação da interface; se ocorrer erro de validação ou de escrita, o estado anterior é restaurado integralmente.
unittest	TestCase, main	Framework utilizado em test_timbre.py para estruturar e executar os 19 testes unitários automatizados com asserções (assertEqual, assertRaises, assertIs, subTest).
tempfile	TemporaryDirectory	Cria diretórios temporários isolados em demo.py e test_timbre.py para testar a persistência JSON sem alterar os dados reais do usuário em data/timbre.json.

5. CRUD Utilizado e Persistência de Dados

O sistema implementa um CRUD completo (Create, Read, Update, Delete) gerenciado pela classe Cadastro (no arquivo dados.py) com persistência automática em arquivo JSON local (data/timbre.json)

após cada operação realizada na interface gráfica.

5.1. Detalhamento das Operações do CRUD

Operação CRUD	Entidades Atendidas	Métodos em dados.py / modelo.py	Comportamento e Regras de Integridade
Create (Criar)	Músicos, Bandas, Vagas e Vínculos de Integrantes	criarMusico(), criarBanda(), criarVaga(), Banda.adicionarIntegrante()	Gera ID inteiro sequencial único (proximo_id, nunca reutilizado após exclusão). Valida campos obrigatórios, idade >= 0, níveis em {1,2,3}, ao menos 1 habilidade por músico e impede instrumentos duplicados no mesmo perfil.
Read (Consultar / Ler)	Músicos, Bandas, Vagas, Integrantes e Oportunidades	Cadastro.carregar(), Busca.executar(), Aplicacao.atualizarListas(), Aplicacao.preencher()	Carrega objetos e reconstitui referências a partir de data/timbre.json; lista todos os registros nas tabelas Treeview; preenche formulários ao selecionar uma linha; filtra vagas compatíveis na aba de busca.
Update (Atualizar)	Músicos, Bandas, Vagas e Estado da Vaga	atualizarMusico(), atualizarBanda(), atualizarVaga(), Vaga.encerrar(), Vaga.reabrir()	Valida todos os novos dados antes de alterar o cadastro. Ao atualizar um músico que já é integrante de bandas, substitui a referência nas bandas preservando o vínculo. Ao atualizar cidade/estilo da banda, reflete imediatamente nas buscas.
Delete (Excluir)	Músicos, Bandas, Vagas e Vínculos de Integrantes	excluirMusico(), excluirBanda(), excluirVaga(), Banda.removerIntegrante()	Integridade referencial automática: excluir um músico remove seu vínculo de todas as bandas; excluir uma banda remove em cascata todas as vagas publicadas por ela; excluir uma vaga a remove da lista interna da banda.

5.2. Mecanismo de Persistência Segura (JSON Atômico + Rollback)

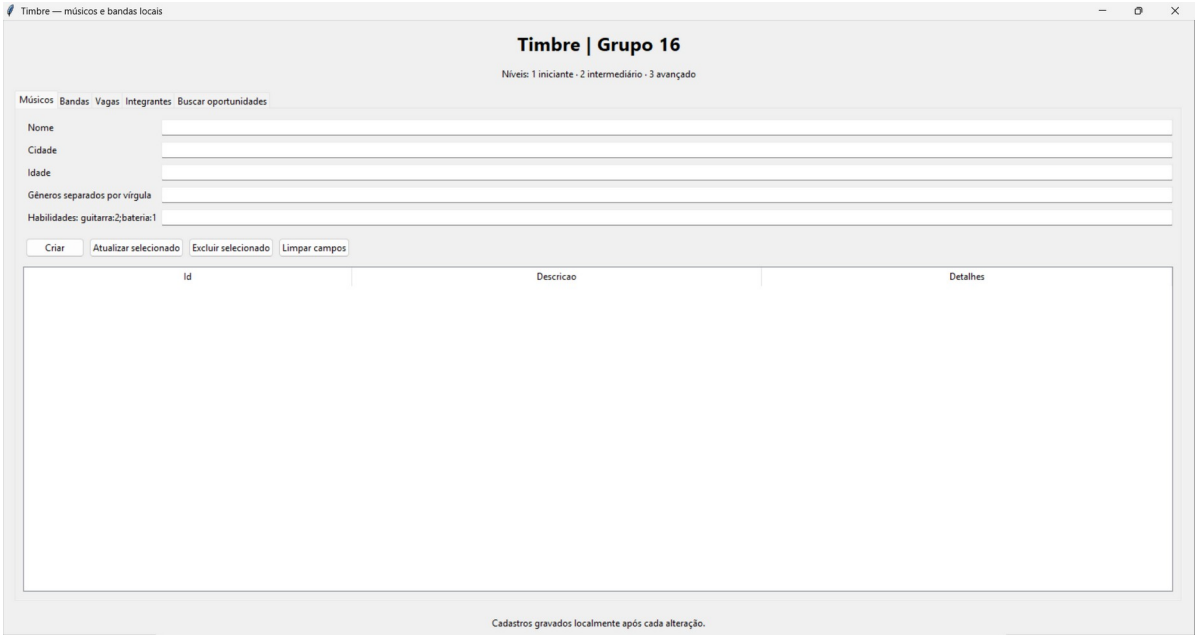
- **Gravação atômica:** Ao salvar em data/timbre.json, Cadastro.salvar() grava primeiro em data/timbre.json.tmp e utiliza os.replace() para substituir o arquivo final de forma atômica, removendo o temporário no bloco finally.
- **Transação em memória (Rollback):** Antes de executar qualquer ação do usuário na interface, Aplicacao.gravar() realiza copy.deepcopy(self.cadastro). Caso ocorra ValueError, KeyError ou OSError, o cadastro em memória é restaurado ao estado anterior e o usuário recebe a mensagem explicativa.
- **Proteção contra perda de dados:** Na abertura do aplicativo, se data/timbre.json não existir, inicia um cadastro vazio; se estiver corrompido ou inválido, exibe um alerta informando o caminho do arquivo e encerra sem sobrescrever o arquivo original.

6. Prints de Telas do Projeto e Explicação de Cada Tela

Abaixo são apresentadas as capturas reais da interface gráfica do Timbre (em execução via python app.py) juntamente com a explicação detalhada do que cada tela e interação representa, tomando como base os registros do documento TIMBRE.docx.

6.1. Anexo 1 — Tela 1: Aba 'Músicos' (Visão Inicial da Interface)

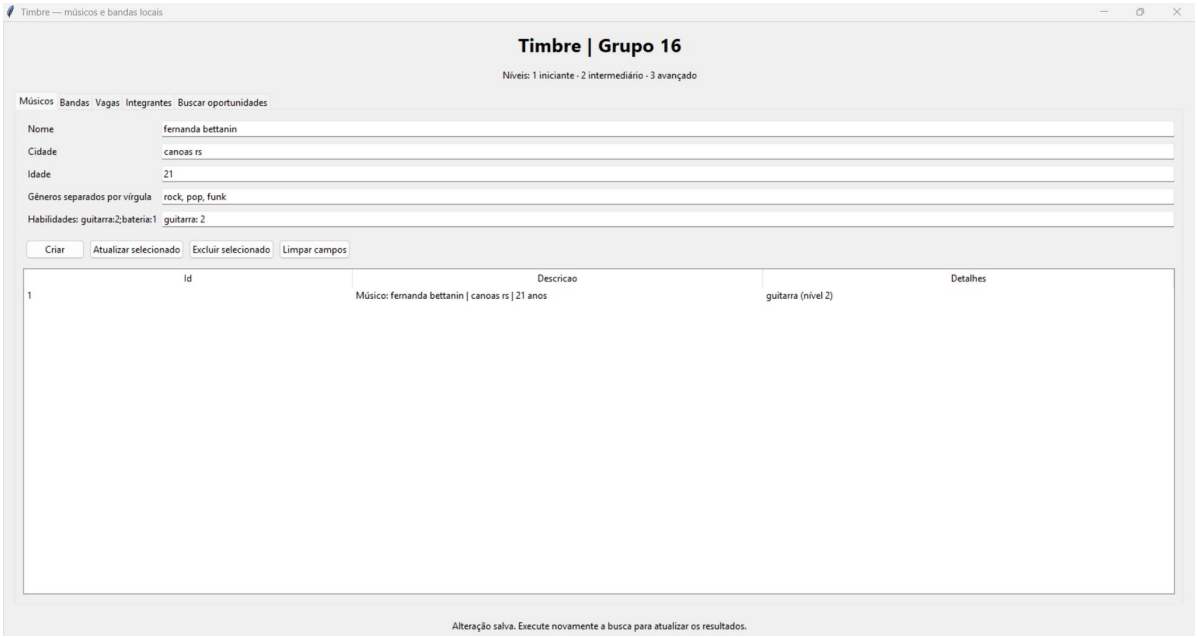
A primeira tela da interface apresenta a guia 'Músicos', contendo os campos de formulário para Nome, Cidade, Idade, Gêneros separados por vírgula e Habilidades no formato 'instrumento:nível' (ex.: guitarra:2;bateria:1), além dos botões de ação ('Criar', 'Atualizar selecionado', 'Excluir selecionado' e 'Limpar campos') e a tabela inferior (Id, Descricao, Detalhes). No topo visualizam-se também as demais abas ('Bandas', 'Vagas', 'Integrantes' e 'Buscar oportunidades') e a legenda dos níveis (1 iniciante · 2 intermediário · 3 avançado). No rodapé, a barra de status informa que os cadastros são gravados localmente após cada alteração.



Anexo 1 — Aba 'Músicos' na inicialização da interface gráfica do Timbre.

6.2. Anexo 2 — Tela 1: Cadastro de Músico Realizado com Sucesso

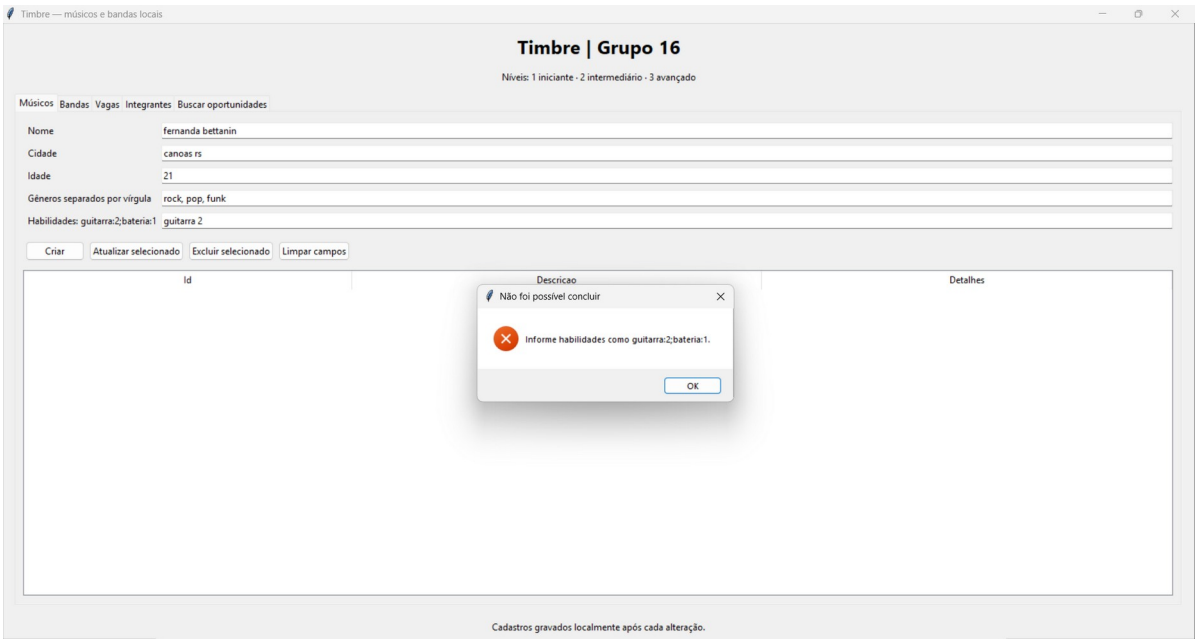
Ainda na aba 'Músicos', esta captura demonstra um cadastro concluído com sucesso após preencher corretamente todos os campos seguindo a formatação indicada: Nome ('fernanda bettanin'), Cidade ('canoas rs'), Idade ('21'), Gêneros ('rock, pop, funk') e Habilidades ('guitarra: 2'). O registro recebe automaticamente o ID 1 na tabela inferior, exibindo o resumo polimórfico 'Músico: fernanda bettanin | canoas rs | 21 anos' na coluna Descrição e 'guitarra (nível 2)' em Detalhes, enquanto o rodapé confirma: 'Alteração salva. Execute novamente a busca para atualizar os resultados.'



Anexo 2 — Cadastro bem-sucedido de músico na aba 'Músicos' com persistência imediata.

6.3. Anexo 3 — Tela 1: Tratamento de Erro ao Omitir Separador de Nível na Habilidade

Demonstra a validação de entrada na aba 'Músicos' quando o usuário tenta cadastrar a habilidade musical sem utilizar o caractere de dois-pontos (':') para separar o instrumento do nível (digitando 'guitarra 2'). O sistema intercepta o erro, preserva o estado do cadastro e exibe uma caixa de diálogo (messagebox) orientando exatamente o formato esperado: 'Informe habilidades como guitarra:2;bateria:1.'

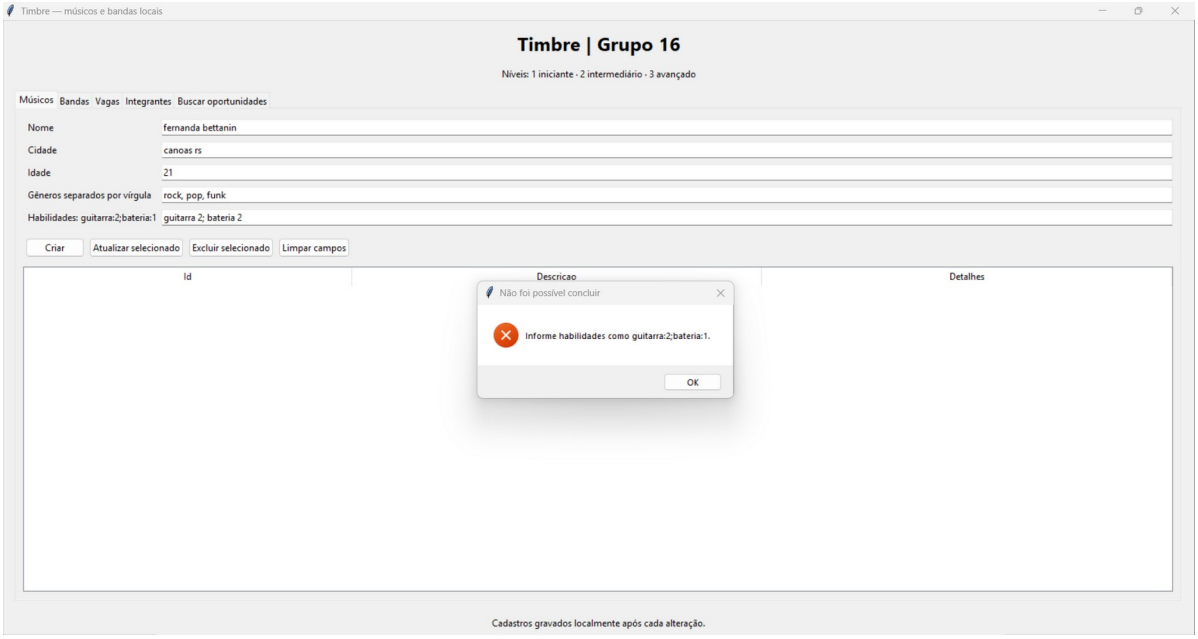


Anexo 3 — Validação impedindo cadastro com formato inválido de habilidade ('guitarra 2').

6.4. Anexo 4 — Tela 1: Tratamento de Erro em Múltiplas Habilidades Fora do Padrão

Demonstra que a validação também atua quando o usuário separa múltiplas habilidades com ponto e vírgula ('guitarra 2; bateria 2'), mas deixa de informar o delimitador ':' antes do nível de cada

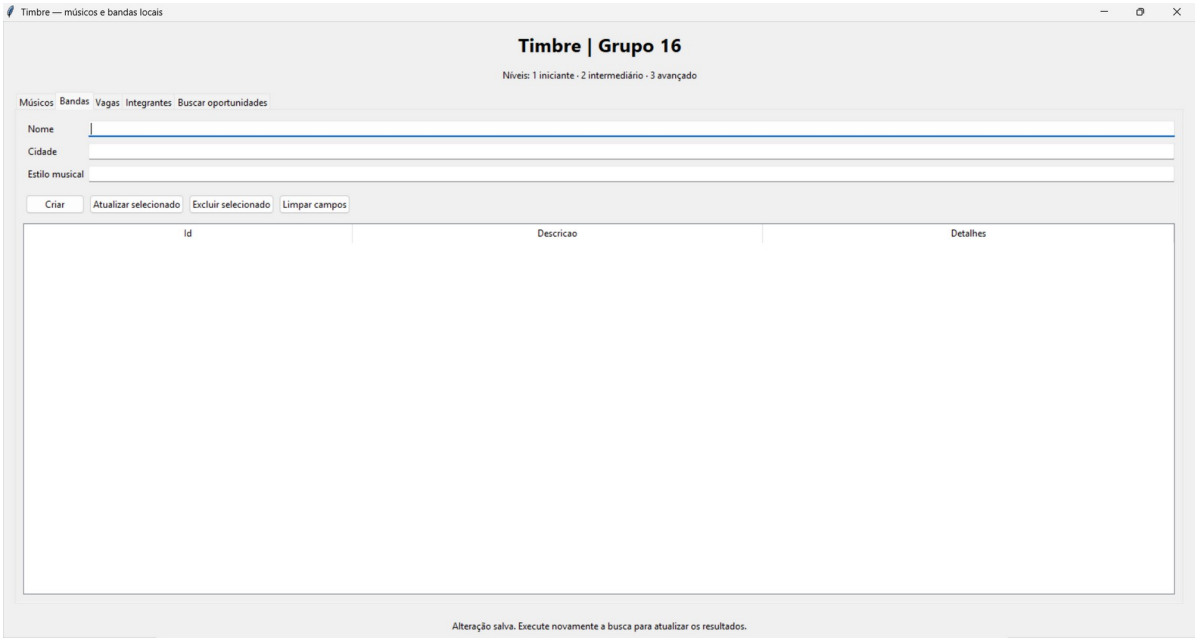
instrumento. Assim como nos casos de nível fora do intervalo permitido (diferente de 1, 2 ou 3), o programa bloqueia o prosseguimento e exibe o alerta de validação correspondente.



Anexo 4 — Validação bloqueando múltiplas habilidades sem o separador ':' de nível.

6.5. Anexo 5 — Tela 2: Aba 'Bandas' (Cadastro e Gerenciamento de Bandas)

Apresenta a segunda aba da interface ('Bandas'), destinada ao cadastro e manutenção de grupos musicais com os campos Nome, Cidade e Estilo musical. Conta com os botões para Criar, Atualizar selecionado, Excluir selecionado (que remove também as vagas associadas à banda) e Limpar campos, além da tabela inferior que lista o ID gerado, o resumo polimórfico da banda e o total de integrantes vinculados.

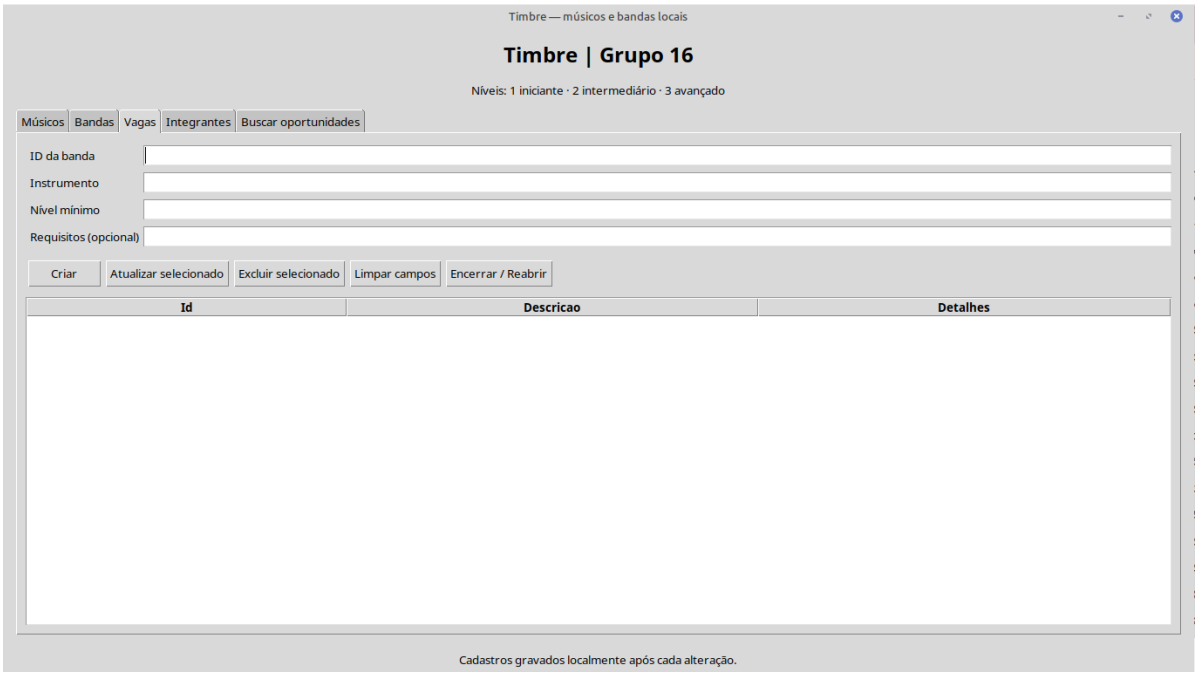


Anexo 5 — Aba 'Bandas' para criação, edição, listagem e exclusão de bandas locais.

6.6. Anexo 6 — Tela 3: Aba 'Vagas' (Publicação e Controle de Disponibilidade)

Apresenta a terceira aba ('Vagas'), na qual uma banda publica oportunidades abertas informando o ID da banda (gerado na aba Bandas), o Instrumento procurado, o Nível mínimo de conhecimento (1 a 3)

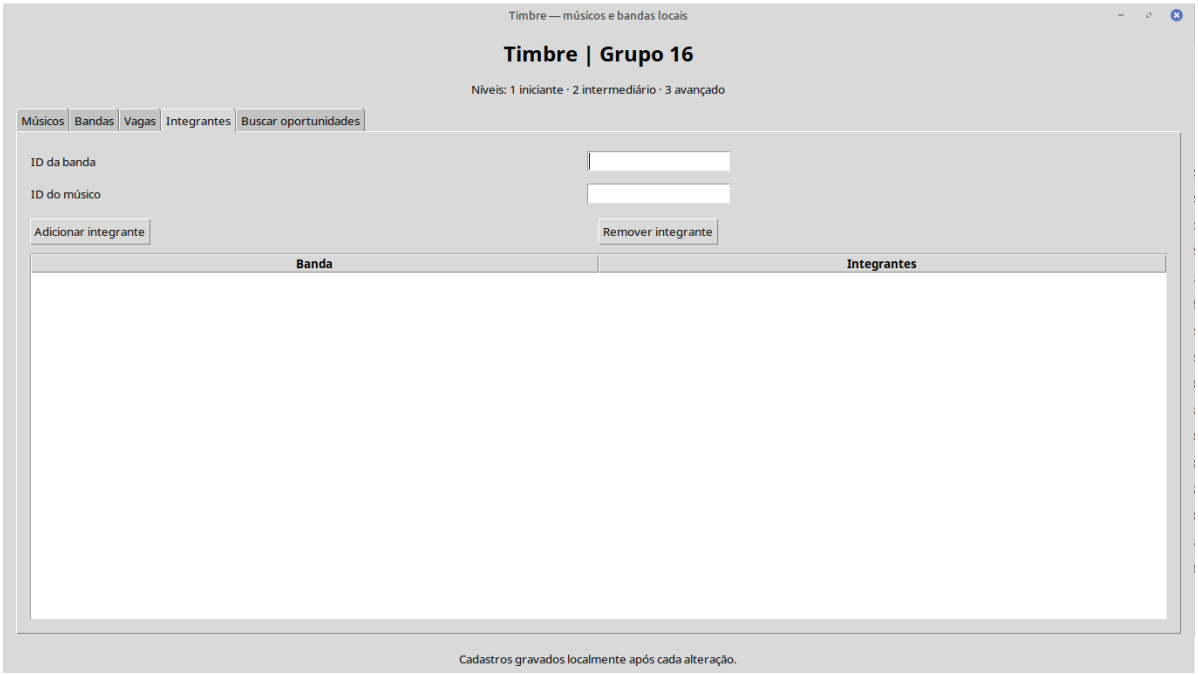
para filtragem automática de compatibilidade e um campo opcional de Requisitos (como disponibilidade de horários ou equipamentos). Além dos botões padrão do CRUD, inclui o botão específico 'Encerrar / Reabrir', que alterna a disponibilidade da vaga selecionada.



Anexo 6 — Aba 'Vagas' com suporte a criação, edição, exclusão e alternância Encerrar / Reabrir.

6.7. Anexo 7 — Tela 4: Aba 'Integrantes' (Associação entre Bandas e Músicos)

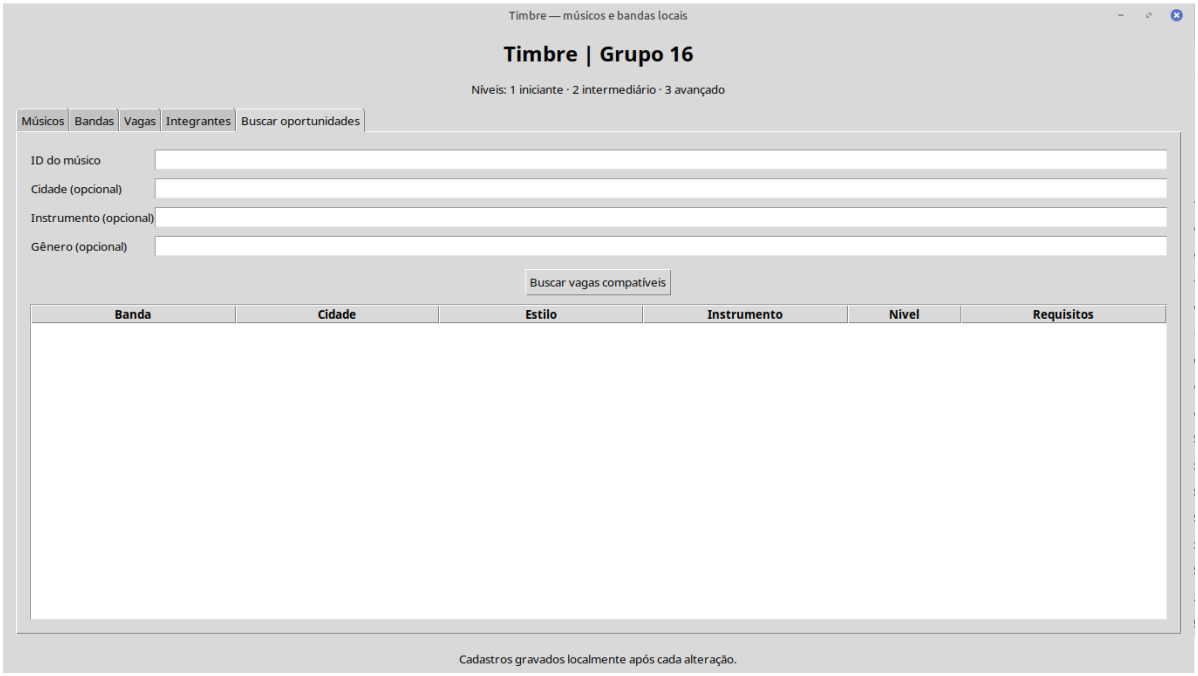
Apresenta a quarta aba ('Integrantes'), responsável por gerenciar a associação N:N (0..* para 0..*) entre bandas e músicos. Contém campos para informar o ID da banda e o ID do músico, acompanhados pelos botões 'Adicionar integrante' e 'Remover integrante'. A tabela inferior exibe cada banda cadastrada e a lista consolidada de todos os músicos que atualmente fazem parte da sua formação.



Anexo 7 — Aba 'Integrantes' para vincular e remover músicos das bandas cadastradas.

6.8. Anexo 8 — Tela 5: Aba 'Buscar oportunidades' (Motor de Compatibilidade e Filtros)

Apresenta a quinta aba ('Buscar oportunidades'), que materializa o objetivo central do Timbre. O usuário informa obrigatoriamente o ID do músico e pode preencher opcionalmente os filtros de Cidade, Instrumento e Gênero. Ao clicar em 'Buscar vagas compatíveis', o sistema executa Busca.executar(), verificando quais vagas estão abertas, exigem um instrumento que o músico toca em nível igual ou superior ao mínimo pedido e atendem aos filtros preenchidos. Os resultados são listados na tabela inferior com as colunas Banda, Cidade, Estilo, Instrumento, Nivel e Requisitos.



Anexo 8 — Aba 'Buscar oportunidades' com filtros combinados e tabela de resultados compatíveis.

7. Exemplos de Testes Realizados

A validação do projeto foi dividida em três frentes complementares: (1) suíte de 19 testes unitários automatizados em test_timbre.py; (2) script de demonstração ponta a ponta no terminal em demo.py; e (3) testes interativos de validação na interface gráfica Tkinter.

7.1. Suíte de 19 Testes Automatizados (test_timbre.py)

Todos os 19 testes automatizados foram executados com unittest e aprovados integralmente (OK em 0.009s). A tabela abaixo resume os cenários testados e os resultados observados:

Método de Teste (test_timbre.py)	Cenário Verificado	Resultado Obtido
test_vaga_compativel_e_nivel_igual_ao_minimo	Vaga aberta para guitarra nível 2 e músico com guitarra nível 2 na mesma cidade/estilo.	Aprovado (OK) — Vaga retornada na busca.
test_nivel_insuficiente	Vaga exige guitarra nível 3, mas o músico possui nível 2.	Aprovado (OK) — Vaga descartada ([]).
test_instrumento_diferente	Vaga pede bateria nível 1, mas o músico só toca guitarra.	Aprovado (OK) — compativelCom() retorna False.

Método de Teste (test_timbre.py)	Cenário Verificado	Resultado Obtido
test_encerrar_e_reabrir	Encerrar vaga compatível e depois reabri-la.	Aprovado (OK) — Deixa de aparecer quando encerrada e volta quando reaberta.
test_filtros_independentes	Filtros de cidade ('Curitiba'), instrumento ('bateria') e gênero ('jazz') divergentes da vaga.	Aprovado (OK) — Vaga descartada em cada filtro.
test_acentos_espacos_e_maiusculas	Banda em 'São Paulo' buscada com ' SAO PAULO ', ' GUITARRA ', ' ROCK '.	Aprovado (OK) — Normalização reconhece equivalência.
test_validacao_de_niveis_e_idade	Tentativa de criar níveis 0, 4, True, '2' e idade negativa (-1).	Aprovado (OK) — Lança ValueError em todos os casos.
test_atualizacao_invalida_preserva_cadastro	Tentativa de atualizar músico com nível 8 ou banda com estilo em branco.	Aprovado (OK) — Lança ValueError e preserva dados originais.
test_criacao_invalida_nao_cadastra	Tentativa de criar músico com nome vazio, sem habilidades ou nível 0.	Aprovado (OK) — Nenhum registro inválido é adicionado.
test_multiplas_habilidades_e_duplicidade	Adicionar 'baixo' nível 3 ao músico e tentar duplicar ' BAIXO '.	Aprovado (OK) — Múltiplas habilidades aceitas; duplicata recusada.
test_atualizar_musico_preserva_associacao	Atualizar dados de um músico que já é integrante de uma banda.	Aprovado (OK) — Banda passa a referenciar a instância atualizada.
test_excluir_musico_remove_vinculos	Adicionar músico duas vezes na banda (sem duplicar) e depois excluí-lo.	Aprovado (OK) — Removido do cadastro e da lista de integrantes.
test_excluir_banda_remove_vagas_sem_afetar_outras	Excluir uma banda que possui vaga publicada existindo outra banda ativa.	Aprovado (OK) — Exclui em cascata apenas as vagas da banda removida.
test_excluir_vaga_atualiza_banda	Excluir uma vaga pelo Cadastro.	Aprovado (OK) — Vaga sai de Cadastro.vagas e de Banda.vagas.
test_atualizar_banda_reflete_na_busca	Alterar cidade e estilo da banda para 'Curitiba' e 'jazz'.	Aprovado (OK) — Busca imediata pelos novos dados encontra a vaga.
test_abstracao_e_polimorfismo	Tentar instanciar Perfil diretamente e chamar resumirPerfil() em Musico e Banda.	Aprovado (OK) — Perfil lança TypeError; subclasses retornam resumos distintos.
test_json_preserva_dados_estados_e_associacoes	Salvar cadastro completo (com integrante e vaga encerrada) em JSON e recarregar.	Aprovado (OK) — Objetos, referências, estado da vaga e proximo_id idênticos.
test_arquivo_ausente_inicia_vazio_e_corrompido_e_preservado	Carregar de caminho inexistente e carregar de arquivo com '{invalido'.	Aprovado (OK) — Inexistente inicia vazio; corrompido lança erro sem sobrescrever.
test_nao_reutiliza_id_apos_exclusao	Excluir vaga e criar nova vaga em seguida.	Aprovado (OK) — Novo ID é estritamente maior que o ID excluído.

7.2. Saída Real da Execução dos Testes e da Demonstração

• Registro de execução de resultados-testes.txt (19 testes unitários):

```
test_abstracao_e_polimorfismo (test_timbre.TestTimbre.test_abstracao_e_polimorfismo)
... ok
test_acentos_espacos_e_maiusculas
(test_timbre.TestTimbre.test_acentos_espacos_e_maiusculas) ... ok
test_arquivo_ausente_inicia_vazio_e_corrompido_e_preservado ... ok
test_atualizacao_invalida_preserva_cadastro ... ok
test_atualizar_banda_reflete_na_busca ... ok
test_atualizar_musico_preserva_associacao ... ok
test_criacao_invalida_nao_cadastra ... ok
test_encerrar_e_reabrir ... ok
test_excluir_banda_remove_vagas_sem_afetar_outras ... ok
test_excluir_musico_remove_vinculos ... ok
test_excluir_vaga_atualiza_banda ... ok
test_filtros_independentes ... ok
test_instrumento_diferente ... ok
test_json_preserva_dados_estados_e_associacoes ... ok
test_multiplas_habilidades_e_duplicidade ... ok
test_nao_reutiliza_id_apos_exclusao ... ok
test_nivel_insuficiente ... ok
test_vaga_compativel_e_nivel_igual_ao_minimo ... ok
test_validacao_de_niveis_e_idade ... ok
----------------------------------------------------------------------
Ran 19 tests in 0.009s — OK
```

• Registro de execução de demo.py (resultado-demo.txt):

```
Músico: Alex | Porto Alegre | 20 anos
Banda: Horizonte | Porto Alegre | rock
Busca inicial: 1 vaga(s)
Após encerrar: 0 vaga(s)
Após reabrir: 1 vaga(s)
JSON recuperado: 1 músico(s), 1 banda(s), 1 vaga(s)
```

8. Limitações e Bugs Conhecidos

Nos 19 cenários de teste automatizados e nos testes de validação de interface realizados pelo grupo, não restaram bugs de perda de dados ou falhas de lógica. As limitações conhecidas desta versão e os pontos de atenção identificados são:

- **Persistência local monousuário (JSON):** Os cadastros são salvos localmente em data/timbre.json sem banco de dados cliente-servidor, portanto não há sincronização em rede nem suporte a duas instâncias gravando simultaneamente no mesmo arquivo.
- **Relacionamento por IDs digitados:** Para vincular uma vaga a uma banda, adicionar integrantes ou executar a busca, o usuário precisa informar o ID numérico exibido nas tabelas das abas correspondentes, em vez de selecionar por nome em uma caixa de combinação (Combobox).
- **Formato textual estrito para habilidades:** O campo de habilidades na aba Músicos exige a sintaxe 'instrumento:nível' separada por ';' (ex.: guitarra:2;bateria:1), conforme demonstrado nos Anexos 3 e 4.
- **Comparação textual sem sinônimos ou geolocalização:** A função chave() normaliza maiúsculas/minúsculas, espaços e acentos, mas não utiliza dicionário de sinônimos nem cálculo de distância geográfica (ex.: 'Canoas' e 'canoas rs' são tratadas como cadeias distintas se escritas de modo diferente).

- **Estilo único por banda e filtro de gênero explícito:** Cada banda armazena um estilo principal em texto; os gêneros cadastrados no perfil do músico são informativos e a restrição por gênero na busca ocorre quando o filtro opcional 'Gênero' é preenchido.
- **Inalterabilidade da banda de uma vaga já criada:** Na edição de uma vaga existente, não é permitido trocar o ID da banda diretamente (o sistema orienta excluir a vaga e criar outra na nova banda).
- **Estrutura de diretórios no repositório:** No repositório GitHub, como os arquivos foram enviados diretamente na raiz da branch main, o comando de testes deve ser executado com 'python -m unittest -v' (ou 'python -m unittest discover -s . -p "test_*.py" -v').

9. O que Falta Fazer

O núcleo funcional exigido para a Etapa 3 (modelagem OO com herança, polimorfismo, encapsulamento e associação, CRUD completo, persistência JSON, interface Tkinter nas 5 abas e bateria de testes) já está implementado e pronto para demonstração. As atividades pendentes e evoluções planejadas são:

- **1. Seletores visuais (Combobox) no lugar de IDs manuais:** Substituir os campos de digitação manual de IDs (em Vagas, Integrantes e Busca) por seletores visuais (ttk.Combobox) exibindo 'ID — Nome', tornando a navegação mais intuitiva.
- **2. Subformulário dedicado para habilidades musicais:** Substituir a entrada textual 'guitarra:2;bateria:1' por um subformulário com seletor de nível (1, 2 ou 3) e botão de adicionar/remover habilidade individualmente.
- **3. Filtro automático pelos gêneros favoritos do músico:** Permitir que a busca considere automaticamente a lista de gêneros de interesse cadastrada no perfil do músico quando o filtro de gênero estiver em branco, além de permitir múltiplos estilos por banda.
- **4. Barras de rolagem e modal de detalhes da vaga:** Adicionar barras de rolagem (ttk.Scrollbar) acopladas às tabelas Treeview e janela de visualização de detalhes completos da banda/integrantes ao dar duplo clique em uma oportunidade encontrada.
- **5. Empacotamento executável e banco SQLite:** Empacotar a aplicação (ex.: via PyInstaller) ou migrar a persistência JSON para SQLite (módulo nativo sqlite3) para suportar acesso concorrente e consultas relacionais.

10. Roteiro para Demonstração Funcional em Aula (05/11)

O grupo está apto a demonstrar tanto a interface gráfica (app.py) quanto a execução automatizada (demo.py e test_timbre.py). Recomenda-se seguir a sequência abaixo (duração estimada de 5 minutos):

- **Passo 1 — Mostrar os conceitos de POO no código (1 min):** Mostrar rapidamente em modelo.py a classe abstrata Perfil (linhas 23–51), a herança em Musico e Banda, o polimorfismo em resumirPerfil(), os atributos privados com __ e o método privado Vaga.__validarNivel() (linha 184).
- **Passo 2 — Executar os testes automatizados e a demo no terminal (1 min):** Executar 'python3 -m unittest -v' e 'python3 demo.py' no terminal para comprovar os 19 testes passando e o fluxo de busca/persistência.
- **Passo 3 — Demonstração interativa na interface Tkinter (3 min):** Executar 'python3 app.py': (a) Na aba Músicos, cadastrar um músico com 'guitarra:2;bateria:1' (e demonstrar também um erro de validação); (b) Na aba Bandas, cadastrar uma banda da mesma cidade e estilo; (c) Na aba Integrantes, vincular o músico à banda; (d) Na aba Vagas, criar uma vaga de guitarra nível 2; (e) Na aba Buscar oportunidades, buscar pelo ID do músico e exibir a vaga compatível; (f) Voltar à aba Vagas, clicar em 'Encerrar / Reabrir' e mostrar que uma vaga encerrada deixa de aparecer na

busca; (g) Fechar e abrir novamente o programa mostrando que todos os dados permaneceram salvos em data/timbre.json.